

Scrum Pipeline & Governance Guide

Single-Team Scrum — Digital Procurement & PLS Seedbed

Prepared by: Project Delivery Team

February 23, 2026

Abstract

This document is the canonical Scrum pipeline and governance guide to be used throughout the development lifecycle of the Digital Procurement & PLS Seedbed project (permissioned blockchain). It defines a single unambiguous meaning for every abbreviation and technical term used in the project, prescribes the step-by-step Scrum pipeline from initial requirement gathering through testing, deployment and maintenance, and establishes obligatory artifact management, traceability, and decision workflows (including architecture decisions).

This guide is intended to be **the project procedural source of truth** (see Section 11 for recommended canonical phrasing and storage). It is normative for all teams, contributors, reviewers, auditors, and stakeholders.

Contents

1	Scope and intent	3
2	Terminology: single meanings (Glossary)	3
3	Standards and normative references	4
4	High-level pipeline: steps and responsibilities	4
4.1	Stage 0 — Discovery & Requirements Intake	4
4.2	Stage 1 — Backlog Triage & Classification	5
4.3	Stage 2 — Backlog Refinement	5
4.4	Stage 3 — Sprint Planning	6
4.5	Stage 4 — Sprint Execution (Development)	6
4.6	Stage 5 — Continuous Integration & Automated Testing	6
4.7	Stage 6 — Pre-Production / Staging Validation	7
4.8	Stage 7 — Release & Deployment	7
4.9	Stage 8 — Production Operations & Maintenance	7
4.10	Stage 9 — Feedback Loop (Metrics & Retrospectives)	8
5	Definition of Ready (DoR) and Definition of Done (DoD)	8
5.1	Definition of Ready (DoR)	8
5.2	Definition of Done (DoD)	8
6	Architecture Decision Record (ADR) template	9
7	Traceability rules and matrix	9

8 Handling new requirements, changes, and emergency fixes	10
8.1 New requirement intake	10
8.2 Change control policy	10
9 Macro vs Micro decisions — governance matrix	10
10 CI/CD gating and enforcement (detailed)	10
11 Artifact repository structure and canonical storage	11
12 Auditability and compliance checklist	11
13 Templates and naming conventions (exact strings)	12
14 Examples: End-to-end flow for a requirement	12
15 Enforcement and exceptions	12
16 Appendix: references	12
17 Revision history	13

1 Scope and intent

This document prescribes the single-team Scrum workflow for converting validated requirements into production software and maintaining it thereafter. It is deliberately prescriptive: each term and abbreviation is assigned one exact meaning (see Section 2) and every artifact and activity is defined so there is no ambiguity in interpretation.

The guide assumes the project will follow a permissioned blockchain architecture with off-chain components. It is compatible with the project's Preliminary SRS (ReqIDs R01–R30) and the backlog mapping produced by the Product Owner.

2 Terminology: single meanings (Glossary)

Each entry below is the exclusive accepted meaning for the term throughout the project documents and communication. Use these forms verbatim in all PRs, ADRs, tickets, and audits.

Term / Abbreviation	Definition (single meaning)
Product Backlog	The single, ordered list of all Product Backlog Items (PBIs). The Product Owner (PO) is the owner of the Product Backlog. No other backlog shall be treated as authoritative.
Product Backlog Item (PBI)	Any unit of work in the Product Backlog. PBIs are labeled as Story , Spike , Enabler , Bug , or Task . Each PBI must have a unique identifier of form PBI-### .
Requirement (ReqID)	A canonical requirement referenced in the SRS, identified as Rxx (e.g., R01). The SRS requirement remains the authoritative requirement text. PBIs implement or satisfy requirements and must reference the originating Rxx values.
Definition of Ready (DoR)	The checklist that a PBI must satisfy before the team may commit it to a Sprint.
Definition of Done (DoD)	The checklist required for a PBI to be considered complete and releasable. DoD is enforced by CI & PO acceptance.
Architecture Decision Record (ADR)	A single file that documents a non-trivial architecture or design decision using the project's ADR template and unique ID (ADR-YYYY-NN).
Spike	A timeboxed PBI to reduce uncertainty; deliverable must be an artifact (prototype, measurement, or decision). Spikes are limited to the timebox defined in the PBI.
Enabler	A PBI whose primary purpose is to create architectural runway, capability, or reduce systemic risk (e.g., CI pipeline, monitoring).
Shariah Board Approval	Formal recorded statement (on-chain or in governance registry) that a PLS product meets the applicable Shariah checklist. This term means an explicit, signed acceptance by the designated Shariah authority for a PBI or ADR.
Canonical Backlog Repository	The project's single authoritative storage for the Product Backlog CSV and its derivatives (recommended name; see Section 11).
Traceability Matrix	CSV mapping that links each ReqID (Rxx) to one or more PBI-### items and test artifacts; stored in a canonical location and required for audits.
Release	A set of PBIs that have completed DoD and passed release acceptance checks; one release is created per deployment window.

Continued

Term / Abbreviation	Definition (single meaning)
SLA / SLO	Service Level Agreement / Service Level Objective. The SLO is a measurable objective the system must meet; SLA is the contractual obligation derived from SLO(s). Both are stored in the operations runbook.
PR	Pull Request to merge code or docs into the canonical branch. Every PR must reference one or more PBI-### and any related ADR-.
CI	Continuous Integration pipeline. The set of automated checks that must pass for a PR to be mergeable.
CD	Continuous Delivery/Deployment. The pipeline that promotes artifacts from staging to production per release policy.

3 Standards and normative references

The project shall use the following standards and guidance as normative references for interpretation of technical and compliance requirements. When those standards evolve, the project documents will record the version used.

- **The Scrum Guide (2020 Edition).** Ken Schwaber and Jeff Sutherland. Defines the official single-team Scrum framework and terminology.
- **W3C Decentralized Identifiers (DID) and Verifiable Credentials (VC).** World Wide Web Consortium (W3C). Governs identity representation and cryptographic credential exchange.
- **Permissioned Blockchain Architecture Patterns.** Hyperledger Fabric documentation. Defines channels, private data collections, and endorsement policies.
- **ISO 20022 Financial Messaging Standard.** International Organization for Standardization. Governs structured payment message formats and mapping.
- **GS1 EPCIS Standard.** GS1 Global. Defines event-based supply chain data representation.
- **NIST Cryptographic and Logging Guidance.** National Institute of Standards and Technology (NIST). Governs secure logging, cryptographic controls, and access management.

Note: Each listed standard is referenced to clarify acceptance criteria in PBIs. Where the SRS (ReqIDs) cites other domain sources (e.g., AAOIFI guidance for PLS), those sources are also normative for the relevant PBIs.

4 High-level pipeline: steps and responsibilities

This section defines the pipeline stages in precise, prescriptive terms. Each stage lists required artifacts, who owns them, gating criteria, and the exact words to use in tickets and PRs.

4.1 Stage 0 — Discovery & Requirements Intake

Purpose: capture business objectives, regulatory constraints, high-risk items (legal, Shariah), and produce canonical ReqID entries.

Inputs: stakeholder interview notes, regulatory memos, SRS draft, pilot partner requirements.

Outputs:

- ReqID entries (form: Rxx) added to the SRS file.
- Prioritized risk list and proposed **MVP Top-10** requirements.
- Intake ticket in the canonical backlog: title format “Intake: Rxx — <short title>”.

Owner: Product Owner (PO) for prioritization; Business Analyst for capture; Legal and Shariah liaison for compliance tagging.

Gate to next stage: PO signs the intake ticket “PO:accepted” and assigns priority (P0/P1/P2), and attaches one or more ReqID references.

4.2 Stage 1 — Backlog Triage & Classification

Purpose: convert raw requirements into candidate PBIs with initial size category and required classification.

Actions:

- Create PBI in the canonical backlog CSV with required fields: PBI-###, Type, Title, Description, AcceptanceCriteria, ReqIDs, NFR_Tags, StoryPoints, Priority, Owner, Sprint, Status, Dependencies, Notes.
- Classify Type as exactly one of: Story, Spike, Enabler, Bug, or Task.
- If unclear, create a **Spike** (timeboxed). Spikes must include explicit *Exit Criteria* and a timebox (calendar days).

Owner: PO to create; Developers to perform quick feasibility review.

Gate (DoR Precheck): PBI must have: Title, Description, at least one Acceptance Criterion, one ReqID reference, and an assigned Owner (or P0-delegate). PBIs lacking any of these fields are “Not Ready”.

4.3 Stage 2 — Backlog Refinement

Purpose: prepare PBIs for Sprint Planning by breaking large PBIs, adding testable acceptance criteria, capturing dependencies, and identifying NFR impacts.

Actions (mandatory for refinement):

- Re-write acceptance criteria in testable form (Gherkin is preferred): **Given / When / Then**.
- Attach or link any ADR drafts that the PBI depends on.
- Add NFR acceptance tests inline (e.g., “Latency: 95th percentile < 2s at X RPS (test reference)”), or create a specific Enabler PBI if the NFR requires dedicated work.
- Estimate story points using team consensus (relative estimation).

Owner: PO and Developers jointly. Meeting cadence: at least once per sprint (mid-sprint) for 10–15% of sprint time.

Gate (DoR): PBI must meet the full DoR checklist in Section 5.

4.4 Stage 3 — Sprint Planning

Purpose: commit to a set of PBIs for the sprint that together form a Sprint Goal.

Actions:

- Team selects PBIs (commitment) that are DoR-compliant.
- Break each PBI into Tasks with owners and hours (not mandatory to store in canonical backlog CSV; tasks can exist in the team's sprint board).
- Reserve explicit capacity for Enablers and Tech Debt (default: 20% of sprint capacity; configurable by team).
- Record Sprint Goal as a single sentence: format "Sprint N Goal: <goal text>".

Owner: Developers (commitment) and PO (prioritization).

Gate: Sprint backlog committed and recorded in the canonical backlog repository with Sprint field populated (e.g., "Sprint 1").

4.5 Stage 4 — Sprint Execution (Development)

Purpose: implement PBIs, produce tested deliverables, create ADRs when architectural choices are needed.

Daily actions:

- Daily Scrum focusing on progress toward the Sprint Goal and impediments.
- PR workflow: every change must be a PR that references the associated PBI-### and any ADRs.
- Code review required: minimum one reviewer (two for security-sensitive PBIs).
- PR gating: the PR must pass CI checks (see Section 4.6).

ADR flow for architectural decisions:

1. If a PBI reveals architectural uncertainty, open a **Spike** PBI (timeboxed).
2. Spike deliverable: a prototype or measured evidence and a proposed ADR draft.
3. ADR draft is reviewed in the next refinement meeting. ADRs are written using the template in Section 6.
4. **Micro decision:** team accepts ADR in a PR merged to `adrs/` (no governance required).
5. **Macro decision:** ADR marked "Requires Governance" must be scheduled for governance review and formal sign-off (PO + Architect + Shariah Board or Regulators as required).

Owner: Developers for implementation; Tech Lead/Architect for ADRs; PO for ensuring business alignment.

4.6 Stage 5 — Continuous Integration & Automated Testing

Purpose: ensure code quality, security, and automated verification before merge or release.

CI suite must include (minimum):

- Unit tests with code coverage reporting.
- Static Application Security Testing (SAST).

- Dependency vulnerability scanning (SCA).
- Integration tests for services touched by the PBI (must be run for PRs that touch those services).
- Linting and style checks.
- Test for NFRs where feasible (e.g., automated performance smoke tests for payment path).

Merge policy: PRs may only be merged when all mandatory CI checks pass. Exceptions create a documented risk ticket and require approval by the Security owner.

Owner: CI Engineer / DevOps for pipeline; Developers for test coverage.

4.7 Stage 6 — Pre-Production / Staging Validation

Purpose: validate integrated system behavior, run full PLS test harness, and check compliance items.

Required staging checks before release:

- Full E2E tests for sprint-scoped critical flows (PO must identify critical flows per sprint).
- PLS test harness scenarios (profit/loss) must pass for PBIs that modify PLS logic.
- Load tests for performance-sensitive PBIs (as defined in NFR tags).
- Security scans (DAST) and penetration test tickets must be green or have tracked remediation PBIs.
- Documentation and runbooks for any production-impacting change.

Owner: QA and SRE; PO verifies acceptance criteria are satisfied in staging.

4.8 Stage 7 — Release & Deployment

Purpose: deploy accepted release to production in a controlled, reversible manner.

Release policy:

- Release can proceed when all sprint PBIs are marked Done and release acceptance checks are green.
- Use a safe deployment pattern: feature-flag gated, canary, or blue/green as appropriate per PBI risk profile.
- Include a rollout/rollback plan and a runbook. Document the release in the release notes file and tag the release in Git.

Owner: DevOps (execution), PO (final go/no-go), Security/Compliance (signoffs if required).

4.9 Stage 8 — Production Operations & Maintenance

Purpose: operate the system under SLOs, manage incidents, and feed operational learnings back to backlog.

Operational rules:

- Monitoring: Grafana dashboards for key metrics (throughput, latency, error rate, node health).
- Alerts: SRE on-call must be notified per runbook thresholds.

- Incident handling: incidents open a ticket linked to related PBI(s) or new PBI(s) for remediation. Severity classification must follow the incident policy document.
- Continuous improvement: regular review of SLOs and technical debt, and creation of PBIs for long-term fixes.

Owner: SRE / Ops; PO for prioritization of operational PBIs.

4.10 Stage 9 — Feedback Loop (Metrics & Retrospectives)

Purpose: use data to re-prioritize the backlog and improve team processes.

Actions:

- Sprint Review: PO demonstrates Done PBIs and records stakeholder feedback in canonical backlog.
- Sprint Retrospective: team documents process improvements; modernize DoD/DoR if necessary; create PBIs for improvements.
- Metric review cadence: weekly for operational metrics; monthly for business KPIs.

Owner: PO and Scrum team.

5 Definition of Ready (DoR) and Definition of Done (DoD)

These checklists are enforced as part of Sprint Planning and CI gating. Use exact language below when marking PBIs ready/done.

5.1 Definition of Ready (DoR)

A PBI is **Ready** when all of the following are true:

1. Title and short description use the canonical formats (Title: short human summary; Description: elaborated context and user value).
2. At least one associated ReqID (Rxx) is listed in the AcceptanceCriteria field.
3. One or more explicit, testable Acceptance Criteria are included (prefer Gherkin).
4. Dependencies listed and owners identified.
5. Story Points estimated by the team (relative sizing).
6. Type is one of: Story, Spike, Enabler, Bug, Task.
7. Security/Compliance tag present if PBI touches regulated areas (e.g., KYC, PLS contracts).
8. PO has assigned priority and a PO-approval comment is present in the PBI record.

5.2 Definition of Done (DoD)

A PBI is **Done** when all of the following are verified:

1. Code committed and merged into the canonical main branch (PR closed) referencing the PBI.
2. All CI checks passed (unit tests, SAST, SCA, integration tests required).
3. Acceptance Criteria verified in staging and annotated in the PBI with evidence links (test run IDs, artifacts).

4. Documentation updated (API docs / runbooks / README) where applicable and linked.
5. If PBI is production-impacting:
 - Monitoring dashboards created and alerts configured.
 - Runbook entry for incident handling added.
6. Security: no critical/high vulnerabilities open for the component (or remediation PBIs created and tracked).
7. PO has marked the PBI as Accepted in the backlog with a clear acceptance comment.

6 Architecture Decision Record (ADR) template

Use this exact template for every ADR. Store each ADR as `adrs/ADR-YYYY-NN.md` in the repository. The ADR is the canonical, auditable record of decisions.

```

Title: <short title>
ADR-ID: ADR-YYYY-NN
Status: Proposed | Accepted | Deprecated
Context:
  - Describe the context and forces (constraints, ReqIDs, stakeholders)
Decision:
  - Describe the chosen option in clear, unambiguous language.
Consequences:
  - Positive consequences (benefits)
  - Negative consequences (risks, trade-offs)
  - Migration strategy (if applicable)
Related PBIs:
  - PBI-### (implementing this ADR)
Related ReqIDs:
  - Rxx, Ryy
Decision Date: YYYY-MM-DD
Author: Name / Role
Reviewers: Name / Role, ...
Governance Required: Yes | No # If yes, list governance body (PO,
  Architect, Shariah Board etc.)
Links:
  - Prototype repo, test results, performance graphs, meeting minutes
Signature:
  - For Accepted: record names/roles and date of acceptance

```

7 Traceability rules and matrix

Traceability is mandatory for audit and regulatory compliance. The Traceability Matrix maps ReqID (Rxx) to one or more PBI-### rows and to test artifacts.

Traceability CSV columns (mandatory): ReqID, PBI_IDs (comma), Test_Artifacts (links), ADR_IDs (if any), Owner, Notes

Rules:

1. Every PBI that implements or affects a requirement must include the originating ReqID in the PBI record (AcceptanceCriteria or Notes).
2. Test artifacts (CI job(run id), Gherkin scenario file, load test report) must be linked from the PBI.

3. The Traceability Matrix file must be updated as part of PBI completion and merged by PR.
4. For regulatory or Shariah-impacting PBIs, the traceability record must include an explicit Shariah Board Approval link or regulator sign-off artifact before the PBI is considered Done.

8 Handling new requirements, changes, and emergency fixes

8.1 New requirement intake

Follow the Stage 0 and Stage 1 process. Label all newly created PBIs with the source channel (e.g., “RegulatorRequest”, “Incident”, “ShariahBoard”, “CustomerFeedback”).

8.2 Change control policy

- Minor changes (no cross-cutting or compliance impact) are handled as normal PBIs and scheduled by the PO.
- Major changes (affecting ReqIDs with compliance or Shariah implications) must create a PBI and a corresponding ADR (if architectural) and must not be merged to main without governance sign-off.
- Emergency fixes: create an emergency PBI, open an incident ticket, and follow emergency release playbook. Emergency merges must be followed by a retrospective and an ADR if the emergency reveals architectural deficits.

9 Macro vs Micro decisions — governance matrix

This table defines where decisions are made, who decides, and the required artifacts.

Decision Type	Example	Decision Authorities	Artifact Required	Re-
Micro	API naming, small refactor	Dev Team (Tech Lead)	ADR (short) or PR description	
Mid	Data model change, module boundaries	Tech Lead + PO	ADR + Spike results + Test plans	
Macro	Platform selection, consortium bylaws, Shariah rulings	Governance Board (PO, Architect, Legal, Shariah as needed)	ADR + cost/impact analysis + formal sign-off record	
Emergency	Production P0 fix	SRE + Tech Lead + PO (rapid)	Incident ticket + postmortem + ADR if design change	

10 CI/CD gating and enforcement (detailed)

CI Pipeline stages (required for all PRs):

1. Checkout + Build
2. Unit tests (fail on coverage below threshold for module)

3. Lint + Static analysis
4. SAST (block on Critical)
5. Dependency scan (SCA) — block on Critical CVEs
6. Integration tests (selected by changed components)
7. Automated acceptance tests (Gherkin scenarios tagged by PBI)
8. Deploy to ephemeral staging and smoke test (for PRs that change infra)

CD Pipeline (release):

1. Deploy to staging and run full test suite
2. Performance regression tests (for changes touching critical paths)
3. Security DAST (for web endpoints)
4. Release approval gates (PO, Security, Compliance signoffs)
5. Promote to production using feature toggle / canary / blue-green

11 Artifact repository structure and canonical storage

The recommended canonical names and locations (store in Git repo; protect main branch):

- `backlog/digital_procurement_backlog.csv` — the canonical Product Backlog CSV (the project’s operational **source of truth**).
- `docs/SRS_prelim.tex` — SRS LaTeX source (authoritative requirement document).
- `docs/DoD_DoR.tex` — DoD / DoR canonical checklist (this document compiled).
- `adrs/ADR-YYYY-NN.md` — ADR records (one per file).
- `diagrams/*.mmd, *.puml` — source diagrams (Mermaid / PlantUML).
- `openapi/*.yaml` — API specification (OpenAPI).
- `tests/pls_harness/*` — automated PLS scenarios and CI harness.
- `ops/runbooks/*` — runbooks, release playbooks, SLO definitions.

Preferred phrase for ‘source of truth’: use the phrase **Canonical Backlog Repository** in place of ambiguous terms like “source of truth”. This phrase specifies both the content (the backlog) and its repository location, reducing ambiguity in requests and audits.

12 Auditability and compliance checklist

For any release that affects compliance areas (KYC/AML, PLS, privacy), ensure the following artifacts exist and are linked in the release PR:

- Traceability Matrix rows linking ReqIDs to PBIs.
- ADRs impacting compliance logic.
- Shariah Board Approval artifacts (signed minutes or on-chain record).
- Test evidence (PLS harness results, load test reports, DAST/DAST results).
- Exportable ledger bundle for regulator review (if requested), with provenance proof.

13 Templates and naming conventions (exact strings)

Use the following exact naming conventions:

- PBI IDs: PBI-`{3-digit decimal}` (e.g., PBI-001).
- Requirement IDs: R`{2-digit}` (e.g., R01).
- ADR IDs: ADR-YYYY-NN.
- Branches for feature work: `feature/PBI-###-short-desc`.
- PR title format: PBI-###: `short-title (type)` e.g., PBI-012: Issue tokenized invoice (Story).
- Release tags: `release/vMAJOR.MINOR.PATCH`.

14 Examples: End-to-end flow for a requirement

Example: SRS requirement R07 (PLS contract model).

1. Discovery: PO creates intake note referencing R07.
2. Backlog Triage: create PBI-011 (Spike) and PBI-013 (Story: Deploy PLS contract).
3. Spike executes and produces prototype + ADR (ADR-2026-01).
4. ADR accepted by Tech Lead and flagged “Requires Shariah Board Approval”.
5. Shariah Board Review scheduled; once approved, PBI-013 moves to Sprint Planning.
6. Development proceeds, PR references PBI-013 and ADR-2026-01. CI passes.
7. PBI-013 acceptance tests run (PLS harness, distribution audit). Evidence linked in the PBI.
8. Release: PBI marked Done; traceability matrix updated linking R07 to PBI-011, PBI-013 and ADR-2026-01.

15 Enforcement and exceptions

- This guide is normative. Deviations must be documented in an ADR marked “Process Exception” and require PO + Scrum Master approval.
- Emergency deviations follow the incident policy, and must be followed by a postmortem and appropriate ADRs/PBIs to remediate root causes.

16 Appendix: references

The following normative references are used in acceptance criteria and compliance interpretations:

- **The Scrum Guide (2020 Edition)**, Ken Schwaber and Jeff Sutherland. Defines the Scrum framework for single-team development.
- **W3C Decentralized Identifiers (DID) v1.0 and Verifiable Credentials Data Model v1.1**, World Wide Web Consortium (W3C).
- **Hyperledger Fabric Documentation**, Linux Foundation. Defines permissioned blockchain architecture patterns including channels and private data collections.

- **ISO 20022 Financial Services Message Scheme**, International Organization for Standardization.
- **GS1 EPCIS and CBV Standard**, GS1 Global. Defines supply chain event capture format.
- **NIST Special Publications (e.g., SP 800-53, SP 800-92)**, National Institute of Standards and Technology. Defines cryptographic, logging, and access control guidance.

17 Revision history

Date	Version	Summary of changes
February 23, 2026	1.0	Initial canonical Scrum pipeline & governance guide (this document).

Usage note: Keep this LaTeX source under `docs/` in the project Git repository. Any change to this guide must be made by Pull Request and requires PO & Tech Lead approval. This file is the operational policy for how the team executes Scrum and manages architectural and compliance decisions.